OPEN SOURCE SOFTWARE PROJECT REPORT

Title: Open Source Audit – Git

Name: Sakshi Rathi

Registration Number: 24BCE10142

Course: Open Source Software (OSS)

Date of Submission: 31 March 26

INTRODUCTION

This project is an audit of the open-source software Git. The main aim of this project is to understand how Git works, why it was created, and how it is used in real-world software development.

The report covers different topics such as the origin of Git, its license, ethical aspects, how it works in Linux systems, and its role in the open-source ecosystem. It also includes a comparison between open-source and proprietary software.

Along with the theoretical concepts, I also performed practical work using shell scripts. This helped me understand Linux commands and basic automation in a better way.

TOOLS USED

- Git
- Linux / Online Bash Environment
- GitHub

PART A — ORIGIN AND PHILOSOPHY

A1: Origin Story of Git

Git is a distributed version control system. It was developed by Linus Torvalds in 2005.

It all started when the Linux kernel community decided to stop using their previous version control system, BitKeeper. This happened because BitKeeper had licensing restrictions and was not fully open-source.

Linus Torvalds wanted to create a version control system that could handle large projects efficiently. He focused on making it fast, reliable, and capable of allowing multiple developers to work together without conflicts.

That is how Git was created. It includes important features such as distributed development, version control, and high performance.

I found it interesting how a real-life problem led to the creation of such an important tool.

A2: License of Git

Git is released under the GNU General Public License version 2 (GPLv2), which is a widely used open-source license. It allows users to use, modify, and distribute the software freely.

The GPL license is based on the idea of freedom in software. It ensures that the software remains open for all users. If someone makes changes to the software and distributes it, they must also share it under the same license.

Open-source software provides users with certain freedoms. These are known as the four basic freedoms:

1. The freedom to use the software for any purpose.

2. The freedom to study how the software works and modify it.

3. The freedom to redistribute copies.

4. The freedom to improve the software and share those improvements.

These freedoms make open-source software different from proprietary software, where users usually have limited control.

From this project, I understood how licensing plays an important role in keeping software open and accessible to everyone.

A3: Ethics of Open Source Software

The basic idea behind the development of open-source software is to share the knowledge with everyone. This helps promote innovation.

From an ethical perspective, open-source software is good because it helps people understand how the software is made. In addition, it is free, which helps people modify it according to their needs. This is especially helpful for students who want to learn without having to pay.

However, there are certain drawbacks. For example, not all software can be made open-source, especially if companies invest heavily in their software. In that case, they would not want their software to be made open-source.

In my opinion, it is necessary to maintain a balance. On the one hand, it is necessary to promote open-source software so that people can learn and grow. On the other hand, it is necessary to allow companies to keep their software proprietary if they wish.

Overall, it is necessary to promote open-source software, especially for the development of technology.

PART B — LINUX FOOTPRINT

Git works well with the Linux environment and is commonly used by developers for version control. It can be easily installed and operated using the terminal.

Git can be installed using various package managers. For example, in Ubuntu, it is installed using the following command:

sudo apt install git

After installing Git, it can be checked using the following command:

git --version

Git stores all the data in a hidden folder named .git, which is automatically created after using the following command to initialize the repository:

git init

Some of the commonly used Git commands in the Linux environment are as follows:

- git init → to initialize the repository
- git add → to add files
- git commit → to save changes
- git status → to check the status
- git log → to check the history

Git works well in the Linux environment because it is lightweight, fast, and easy to use through the terminal.

From this part, I understood how to install and use Git in a Linux environment.

PART C — FOSS ECOSYSTEM

Git plays an important role in the open-source world and is widely used along with many other tools and platforms.

There are platforms like GitHub and GitLab that use Git for storing and managing code. These platforms allow developers to upload their projects online and collaborate with others from different locations.

One of the main reasons Git is so useful is because it supports teamwork. Features like branching and merging make it possible for multiple developers to work on the same project without disturbing the main code.

The open-source community also contributes a lot to this ecosystem. Developers from different parts of the world work together to improve software, fix bugs, and add new features.

Git can also be used with other tools such as automation tools and code review systems, which makes the overall development process easier and more organized.

From this part, I understood how Git works with different tools and how it helps developers collaborate in real-world projects.

PART D — OPEN SOURCE VS PROPRIETARY

Open-source and proprietary software follow different development and licensing models. The following comparison highlights the key differences between the two approaches.

Dimension	**Open Source**	**Proprietary Alternative**
Cost	Completely free. No license fees for any team size.	Free for limited users; enterprise versions are expensive.
Security (who can audit the code?)	Source code is open and can be audited by anyone.	Closed source; only the company can audit internally.
Support and Reliability	Community support through forums and mailing lists; no guaranteed SLA.	Professional paid support with guaranteed service.
Freedom to Modify	Users can modify, share, and customize the software freely.	Users cannot modify the software.
Community vs Corporate Control	Developed and maintained by the open-source community.	Fully controlled by the company.

Dimension	Open Source	Proprietary Alternative
Overall Verdict	Best choice for most developers due to flexibility and cost.	Suitable mainly for specific enterprise use cases.

PART E — SHELL SCRIPTS

Script 1: System Identity Report

code:

```
#!/bin/bash

echo "===== System Identity Report =====

# Display kernel version of the system

echo "Kernel Version: $(uname -r)"

# Show the currently logged-in user

echo "Current User: $(whoami)"

# Show system uptime

echo "System Uptime: $(uptime -p)"

# Display current system date and time

echo "Current Date: $(date)"

# Pause so the user can see the output

read -p "Press Enter to exit..."
```

Output:

```
saksh@Victus MINGW64 ~
$ ./script1.sh
===== System Identity Report =====
Kernel Version: 3.6.5-22c95533.x86_64
Current User: saksh
System Uptime: Information not available in this environment
Current Date: Tue Mar 31 02:33:24 IST 2026
Press Enter to exit...
```

Script 2: FOSS Package Inspector

Code:

```
#!/bin/bash
echo "----- Log File Analyzer -----"
# Check if user provided both arguments
if [ $# -lt 2 ]
then
   echo "Usage: $0 <file_path> <keyword>"
   exit 1
fi
# Store input arguments
file=$1
keyword=$2
count=0
# Verify file exists before processing
if [ ! -f "$file" ]
then
   echo "File not found."
   exit 1
fi
# Read file line by line and search keyword
while read -r line
do
   if echo "$line" | grep -i "$keyword" > /dev/null
   then
```

```
        count=$((count+1))
    fi
done < "$file"
# Print final count
echo "The keyword '$keyword' appeared $count times in the file."
```

Explanation:

Output:

```
saksh@Victus MINGW64 ~
$ bash script2.sh
===== Git Installation Check =====
Git is installed on this system.
Installed Git version:
git version 2.52.0.windows.1
===check completed===
```

Script 3: Disk and Permission Auditor

Code:

```
#!/bin/bash
# Print heading for the report
echo "----- Directory Audit Report -----"
echo ""
# Loop through selected directories (/tmp and current directory)
for dir in /tmp .
do
    # Check if the directory actually exists
    if [ -d "$dir" ]; then
```

```
        # Display which directory is currently being checked
        echo "Checking Directory: $dir"
        # Get the total size of the directory
        # 'du -sh' shows the size in human readable format
        dir_size=$(du -sh "$dir" 2>/dev/null | cut -f1)
        echo "Directory Size: $dir_size"
        # Get the directory permission information
        # 'ls -ld' shows details of the directory itself
        dir_perm=$(ls -ld "$dir" | awk '{print $1}')
        echo "Directory Permissions: $dir_perm"
        echo ""
    else
        # Message shown if directory does not exist
        echo "Directory $dir not found."
    fi
done
# Final message indicating completion
echo "----- Audit Completed -----"
```

Explanation:

This script checks different directories and displays their size and permissions.

It uses a loop to go through each directory and verifies whether it exists. The "du" command is used to calculate the size of the directory, while "ls -ld" is used to display its permissions.

While working on this script, I understood how loops and conditions can be used together to handle multiple directories in shell scripting.

Output:

```
saksh@Victus MINGW64 ~
$ bash script3.sh
----- Directory Audit Report -----

Checking Directory: /tmp
Directory Size: 378M
Directory Permissions: drwxr-xr-x

Checking Directory: .
```

Script 4: Log File Analyzer

Code:

```
#!/bin/bash

# Print heading

echo "----- Log File Analyzer -----"

# Check if the user provided at least 2 arguments

# (file path and keyword)

if [ $# -lt 2 ]; then

    echo "Usage: $0 <file_path> <keyword>"

    exit 1
```

```
fi
# Store arguments in variables
file=$1
keyword=$2
count=0
# Check if the specified file exists
if [ ! -f "$file" ]; then
    echo "File not found."
    exit 1
fi
# Read the file line by line
# 'IFS=' prevents trimming spaces
# '-r' prevents backslash interpretation
while IFS= read -r line
do
    # Check if the current line contains the keyword
    # '-i' makes the search case-insensitive
    if echo "$line" | grep -i "$keyword" > /dev/null; then
        # Increase count if keyword is found
        count=$((count+1))
    fi
done < "$file"
```

Print the final count

echo "The keyword '$keyword' appeared $count times in the file."

Explanation:

This script analyzes a log file and counts how many times a given keyword appears.

It first checks whether the user has provided the required inputs (file name and keyword). Then it verifies if the file exists. After that, it reads the file line by line using a while loop and checks for the keyword using the grep command.

If the keyword is found, the counter increases. Finally, the total count is displayed.

While working on this script, I understood how file handling, loops, and conditions are used together in shell scripting.

Output:

```
saksh@Victus MINGW64 ~
$ bash script4.sh
----- Log File Analyzer -----
Usage: script4.sh <file_path> <keyword>

saksh@Victus MINGW64 ~
```

Script 5: Open Source Manifesto Generator

Code:

#!/bin/bash

Print a friendly welcome message to the user

echo "----- Open Source Manifesto Generator -----"

```
# Prompt the user to enter their name
# This will be stored in a variable for later use
read -p "Enter your name: " name
# Prompt the user to enter their favorite open-source tool
# Again, we'll store this input for the manifesto
read -p "Enter your favorite open-source tool: " tool
# Define the name of the file where the manifesto will be saved
file_name="manifesto.txt"
# Inform the user that the file is being created
echo "Creating your manifesto..."
# Start writing the manifesto into the file
# The first line introduces the user
echo "Hello, my name is $name." > $file_name
# Add a line expressing support for open-source software
echo "I believe in the power of open-source software." >> $file_name
# Include the user's favorite tool in the manifesto
echo "My favorite tool is $tool." >> $file_name
# Add a line about collaboration and knowledge sharing
echo "I support collaboration and sharing of knowledge." >>
$file_name
# Final message confirming the manifesto has been created
echo "Manifesto created successfully in file: $file_name"
```

Optional: You could also print the manifesto to the terminal

Uncomment the next line if you want to see it immediately

cat $file_name

Explanation:

This script is used to generate a simple text file based on user input.It takes input such as the user's name and favorite open-source tool. Then it creates a file and writes a short message into it using echo commands.

While working on this script, I understood how to take input from users and create files using shell scripting.

Output:

```
C:\Users\saksh>bash script5.sh
----- Open Source Manifesto Generator -----
Enter your name: sakshi
Enter your favorite open-source tool: git
Creating your manifesto...
Manifesto created successfully in file: manifesto.txt
```

```
saksh@Victus MINGW64 ~
$ bash script5.sh
----- Open Source Manifesto Generator -----
Enter your name: Sakshi Rathi
Enter your favorite open-source tool: git
Creating your manifesto...
Manifesto created successfully in file: manifesto.txt
```

CONCLUSION

This project helped me understand the importance of open-source software and how Git is used in real-world development. I also gained practical experience in Linux and shell scripting, which improved my overall understanding of the subject.